WEEK3TASK

# Back-End API Development

StudySync: a Flask REST API for collaborative study groups

| Item | Details |
|---|---|
| Prepared by | Divyanshi Katiyar |
| Programme | Junior Full Stack Developer Internship, Week 3 |
| Project | StudySync (Week 1: planning and architecture; Week 2: React front end) |
| Stack | Python 3, Flask 3, SQLite, PyJWT, scrypt password hashing, unittest |
| Source code | https://github.com/divyanshi-katiyar/StudySync (folder studysync-server) |
| Deliverable | ZIP with source code, README, API documentation (Markdown and OpenAPI), tests and benchmark scripts; this report |
| Date | 25 October 2026 |

## Summary

This week I built the back end that the Week 2 React application was designed to call: 42 REST endpoints covering sign-up and sign-in, study groups with owner, moderator and member roles, shared resources, discussion threads, scheduled sessions with RSVPs, notifications and a dashboard. Every response uses the JSON shapes the front end already consumes, so switching the front end from its mock API to the real one is a configuration change plus one small set of replacement files.

| Measure | Result |
|---|---|
| Endpoints | 42 (listed in section 6); documented in docs/API.md and docs/openapi.yaml |
| Database | 11 tables, foreign keys enforced, 11 explicit indexes, CHECK constraints on every enumerated column, one-owner-per-group rule enforced by the database |
| Code | about 1,850 lines of application code and 1,040 lines of tests |
| Automated tests | 114 tests, all passing in about 5 seconds (auth 33, groups 31, content 14, sessions 14, security 20, docs 2) |
| Response time (demo data) | median 2.12 to 4.63 ms and 95th percentile at most 5.13 ms for reads, over 200 requests each; the Week 1 target was under 300 ms |
| Response time (larger data) | with 2,008 groups, 18,069 memberships and 10,210 resources: 95th percentile at most 13.0 ms |
| Concurrency | 20 client threads for 10 s: 220.1 requests per second, 0 errors, 95th percentile 167.33 ms |
| Sign-in cost | 111.25 ms on average (scrypt is deliberately slow) |
| Front-end check | the new HTTP client passed a 12-step script against the running server (login, groups, resources, sessions, RSVP, 403 and 422 handling, logout, session restore after reload) |

## What was not verified

I could not re-run the React project's own Vitest suite or a production build for this report, so the front-end changes in **frontend-integration/** were verified only by the Node script above. Run `npm test` and click through the app against the API before submitting (risk R12). The API also has not been deployed to a real host, so cross-site cookie behaviour (risk R10) is tested in configuration but not on live infrastructure.

# 1. Scope and How This Report Responds to Earlier Feedback

The Week 1 reviewer asked for specific performance numbers, concrete benchmarks and a more detailed risk assessment. Those requests apply to this week as well, so this report measures instead of estimating.

| Earlier feedback | Where it is answered |
|---|---|
| Specific examples for performance metrics | Section 10: per-endpoint mean, median, p95 and p99 over 200 requests, a larger-data run, a concurrency run, and sign-in cost |
| Concrete numbers for benchmarks | Section 10 compares every figure with the Week 1 target (under 300 ms); section 9 gives test counts per area |
| More detailed risk assessment | Section 11: 13 risks scored by likelihood and impact (1 to 5) with early warnings, mitigation, evidence and status; open items are stated plainly |

## 1.1 Week 3 requirements and where each is met

| Task requirement | Delivered |
|---|---|
| RESTful APIs with CRUD and a database | 42 endpoints; create, read, update and delete for groups, resources, posts, comments, sessions; SQLite database (section 4) |
| Secure authentication | scrypt hashing, 15-minute JWT access tokens, rotating refresh tokens in an httpOnly cookie, reuse detection, rate limits (section 7) |
| Error handling and data validation | one JSON error shape, 422 with per-field messages, query-parameter validation, generic 500 (section 8) |
| Relational or non-relational schema | relational schema with constraints and indexes (section 4) |
| Tests | 114 automated tests including 20 security tests (section 9) |
| API documentation | docs/API.md (generated, with real example responses), docs/openapi.yaml, and a test that fails if docs and code disagree |
| README with setup and run steps | README.md in the ZIP; commands are repeated in section 12 |

## 1.2 Week 1 functional requirements covered by the API

| Requirement | Endpoints | Status |
|---|---|---|
| FR-01 Register, validate, reject duplicate email | POST /auth/register (409 on duplicates, case-insensitive) | Done |
| FR-02 Log in and out with JWT access and refresh tokens | POST /auth/login, /refresh, /logout | Done |
| FR-03 View and edit profile | GET and PATCH /users/me; POST /users/me/password (extra) | Done |
| FR-04 Create group with visibility | POST /groups | Done |
| FR-05 Discover with search, filter, sort, pagination | GET /groups | Done |
| FR-06 Join public group, join with invite code, leave | POST /groups/:id/join, /join-by-code; DELETE /leave | Done |
| FR-07 Roles owner, moderator, member | PATCH and DELETE /groups/:id/members/:userId; POST /transfer-ownership (extra) | Done |
| FR-08 Resources with tags: list, filter, delete | GET, POST /groups/:id/resources; PATCH, DELETE /resources/:id | Done |
| FR-09 Discussion: posts and comments | posts and comments endpoints, including edit and delete | Done |
| FR-10 Sessions with RSVP | sessions endpoints; PUT /sessions/:id/rsvp | Done |
| FR-11 Notifications | Stored per recipient by the server and read through /notifications. Live push over Socket.io is Week 4 | Partly (stored, not pushed) |
| FR-12 Dashboard | GET /users/me/dashboard | Done |
| FR-13 to FR-15 | Email reminders, admin panel, video calls: Could or Won't in Week 1 | Not in scope |

# 2. Technology Choices and Changes from the Week 1 Plan

Week 1 proposed Node.js, Express and MongoDB. The Week 3 task allows Node with Express, Django or Flask, and relational or non-relational storage. I implemented the back end in **Flask with SQLite**. This is a deliberate change from the plan, so the table below records what changed and why.

| Area | Week 1 plan | Week 3 implementation | Reason and trade-off |
|---|---|---|---|
| Framework | Node.js + Express | Python 3 + Flask 3 (application factory, blueprints) | Listed as allowed in the task. Same layered structure (routes, services, data access). Trade-off: the back end no longer shares a language with the React front end. |
| Database | MongoDB + Mongoose | SQLite (relational), raw parameterised SQL | The data is relational: users and groups are many-to-many, and rules such as one owner per group, cascade delete and unique membership are enforced by the database (partial unique index, foreign keys) rather than only by service code. Needs no server, so anyone can run the tests. Trade-off: SQLite suits one server; PostgreSQL is the next step for several instances (risk R6). |
| Validation | Zod | Small rule functions in validation.py | Messages are identical to the front end's validation.js, so the same error text appears whichever side catches the problem. |
| Password hashing | bcrypt, cost 12 | scrypt (Werkzeug) | Memory-hard, available through Python's hashlib with no extra package; about 111 ms per sign-in measured. |
| Security headers and NoSQL sanitising | Helmet, express-mongo-sanitize | Headers set in one after-request hook; not needed for sanitising | Parameterised SQL removes the injection class that mongo-sanitize addressed in MongoDB. |
| Real-time | Socket.io | Deferred to Week 4 | Notifications are already stored per recipient, so the push layer only needs to emit events. |
| File uploads | Cloudinary | Deferred | As in Week 2, a file resource is a link. |
| Logout access | Signed-in users | Public endpoint | A user whose access token has expired must still be able to end the session and clear the cookie. |
| Extra endpoints | - | Edit and delete for resources, posts, comments, sessions; change password; transfer ownership | Completes CRUD for every entity and implements the Week 1 rule that an owner must transfer ownership before leaving. |

## 2.1 Project layout

```
studysync-server/
  studysync/
    __init__.py       app factory: CORS, security headers, rate limits, request log
    config.py db.py schema.sql security.py validation.py errors.py ratelimit.py
    serializers.py    rows -> camelCase JSON in the shapes the React app uses
    routes/           auth, users, groups, content, sessions (HTTP only)
    services/         auth, groups, content, sessions, notifications (rules + SQL)
    seed.py           demo data identical to the Week 2 mock database
  tests/              114 tests      docs/   spec.py, API.md, openapi.yaml, benchmark.json
  scripts/            gen_docs.py, benchmark.py, client_smoke_test.mjs
  frontend-integration/  files to copy into the React project
  manage.py run.py wsgi.py requirements.txt .env.example README.md
```

# 3. System Architecture

The back end keeps the three-tier shape from Week 1. Every request passes through the same pipeline; routes stay thin, services own the rules, and only serializers decide what the client sees.

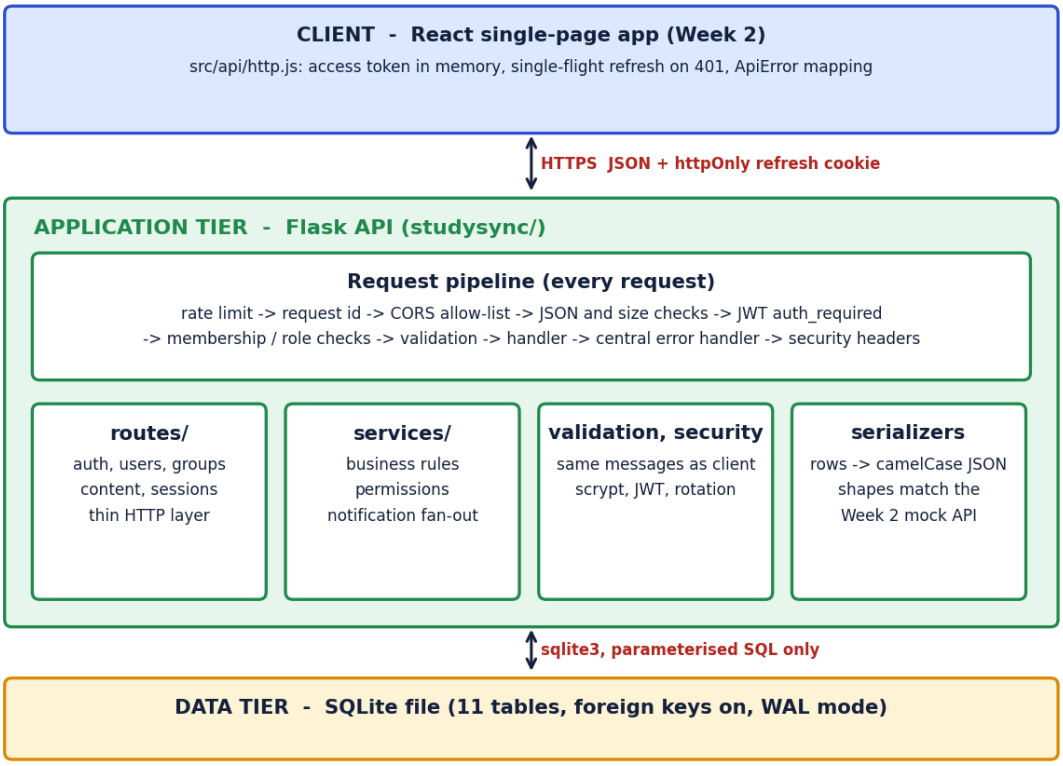



*Figure 1: Back-end architecture. The React client talks to the Flask API over HTTPS; the API talks to SQLite with parameterised SQL only.*

## 3.1 Life of a request: sharing a resource

- The browser sends `POST /api/groups/g1/resources` with the access token in the Authorization header.
- The rate limiter counts the request, a request id is assigned, and the CORS allow-list is applied.
- `auth_required` verifies the JWT signature, expiry and type, then loads the user. Failure returns 401.
- The service checks the caller is a member of the group (403 otherwise; 404 if the group does not exist).
- Validation checks title, type, link scheme (only http and https) and tags. Failure returns 422 with a message for each bad field.
- In one database transaction the service inserts the resource and its tags and writes a notification for every other member.
- The serializer returns the new resource with 201; a final hook adds security headers and the request id.

# 4. Database Design

The Week 1 ER diagram translates to eleven tables. Two things that were embedded arrays in MongoDB become tables: resource tags and session attendees. One table is new: *refresh_tokens*, which stores hashed refresh tokens for session management.

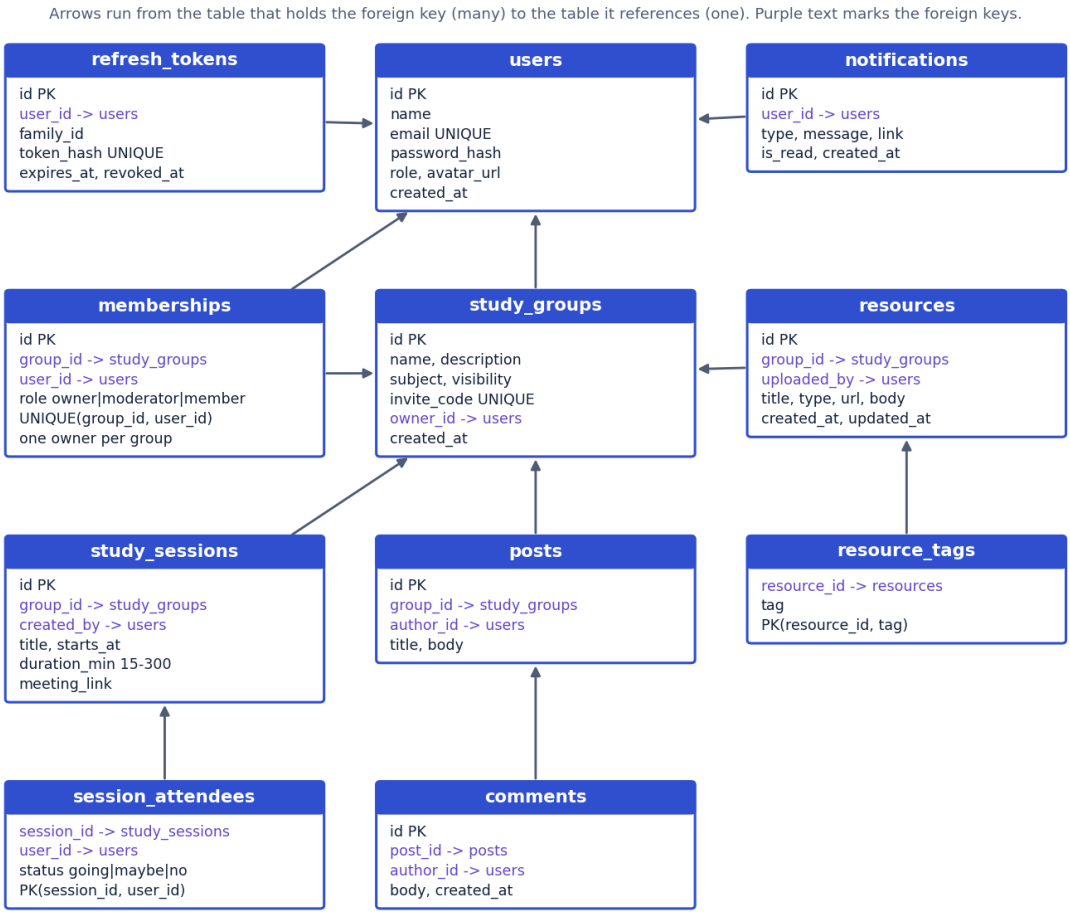



*Figure 2: Entity relationship diagram of the implemented schema.*

## 4.1 Integrity rules enforced by the database

- **Foreign keys are on** (PRAGMA foreign_keys). Deleting a group cascades to memberships, resources, tags, posts, comments, sessions and RSVPs; a test checks that all seven tables are empty afterwards.
- **One owner per group**: a partial unique index on memberships(group_id) where role = 'owner'. A test shows the database itself rejects a second owner.
- **No duplicate membership or RSVP**: unique (group_id, user_id) and primary key (session_id, user_id).
- **CHECK constraints** on role, visibility, resource type, RSVP status, notification type and session length (15 to 300 minutes).
- **Unique** email and invite code. Content authors use ON DELETE SET NULL so history survives a deleted account.
- **Secrets at rest**: password hashes only; refresh tokens stored as SHA-256 hashes, never in the clear.

## 4.2 Indexes

| Index | Serves |
|---|---|
| study_groups(visibility, subject) | Discover filter by subject among public groups |
| memberships(user_id) and unique (group_id, user_id) | My groups; member counts and permission checks |
| resources(group_id, created_at); resource_tags(tag) | Newest-first resource lists; tag filter |
| posts(group_id, created_at); comments(post_id, created_at) | Discussion lists and threads |
| study_sessions(group_id, starts_at) | Upcoming sessions and next session of a group |
| notifications(user_id, is_read, created_at) | Notification feed and unread filter |
| refresh_tokens(family_id), (user_id) | Revoking a token family or all of a user's sessions |

# 5. REST API

Conventions: nouns in URLs, HTTP verbs for actions, JSON bodies, success shape `{ success: true, data }`, error shape `{ success: false, message, errors? }`. Status codes used: 200, 201, 400, 401, 403, 404, 409, 413, 415, 422, 429, 500. Paged lists send the total in the X-Total-Count header; discovery returns { items, total, page, pages }. The complete reference with request fields, rules and real example responses is in docs/API.md and docs/openapi.yaml (import the YAML into Postman or Swagger UI).

## 6. Endpoint Summary (42)

| Method | Endpoint | Access | Purpose |
|---|---|---|---|
| GET | /api/health | Public | Liveness check (also touches the database) |
| POST | /api/auth/register | Public | Create an account and sign in |
| POST | /api/auth/login | Public | Verify credentials and sign in |
| POST | /api/auth/refresh | Refresh cookie | Exchange the httpOnly refresh cookie for a new access token |
| POST | /api/auth/logout | Public | Revoke the refresh token and clear the cookie |
| GET | /api/users/me | Signed in | The current user's profile |
| PATCH | /api/users/me | Signed in | Update name and/or avatar link |
| POST | /api/users/me/password | Signed in | Change password |
| GET | /api/users/me/dashboard | Signed in | Counts, the next 3 sessions and up to 3 of my groups |
| GET | /api/groups | Signed in | Discover public groups (never returns private groups) |
| POST | /api/groups | Signed in | Create a group |
| GET | /api/groups/mine | Signed in | Groups I belong to, sorted by name |
| POST | /api/groups/join-by-code | Signed in | Join any group (including private) with its invite code |
| GET | /api/groups/:group_id | Member (public groups: preview) | Group details |
| PATCH | /api/groups/:group_id | Owner | Edit name, description, subject or visibility |
| DELETE | /api/groups/:group_id | Owner | Delete the group and everything in it (memberships, resources, posts, comments, sessions) |
| POST | /api/groups/:group_id/join | Signed in | Join a public group |
| DELETE | /api/groups/:group_id/leave | Member | Leave a group |
| POST | /api/groups/:group_id/transfer-ownership | Owner | Make another member the owner; the old owner becomes a moderator |
| GET | /api/groups/:group_id/members | Member | Members with roles, owner first |
| PATCH | /api/groups/:group_id/members/:target_id | Owner | Set a member's role to moderator or member |
| DELETE | /api/groups/:group_id/members/:target_id | Owner / moderator | Remove a member |
| GET | /api/groups/:group_id/resources | Member | List resources, newest first |
| POST | /api/groups/:group_id/resources | Member | Share a resource |
| PATCH | /api/resources/:resource_id | Uploader / moderator / owner | Edit title, link or note text, or tags |
| DELETE | /api/resources/:resource_id | Uploader / moderator / owner | Delete a resource and its tags |
| GET | /api/groups/:group_id/posts | Member | List discussion posts, newest first, with comment counts |
| POST | /api/groups/:group_id/posts | Member | Start a discussion |
| GET | /api/posts/:post_id | Member | A post with its group and comments (oldest first) |
| PATCH | /api/posts/:post_id | Author | Edit title or body |

| Method | Endpoint | Access | Purpose |
|---|---|---|---|
| DELETE | /api/posts/:post_id | Author / moderator / owner | Delete a post and its comments |
| POST | /api/posts/:post_id/comments | Member | Reply to a post |
| DELETE | /api/comments/:comment_id | Author / moderator / owner | Delete a comment |
| GET | /api/groups/:group_id/sessions | Member | All sessions of a group in time order (past and future) |
| POST | /api/groups/:group_id/sessions | Moderator / owner | Schedule a session |
| GET | /api/sessions | Signed in | Upcoming sessions across all my groups |
| PATCH | /api/sessions/:session_id | Moderator / owner | Edit a session |
| DELETE | /api/sessions/:session_id | Moderator / owner | Delete a session and its RSVPs |
| PUT | /api/sessions/:session_id/rsvp | Member | Set my RSVP (idempotent) |
| GET | /api/notifications | Signed in | My notifications, newest first |
| PATCH | /api/notifications/read-all | Signed in | Mark all my notifications as read |
| PATCH | /api/notifications/:notification_id/read | Signed in | Mark one notification as read |

# 7. Authentication and Security

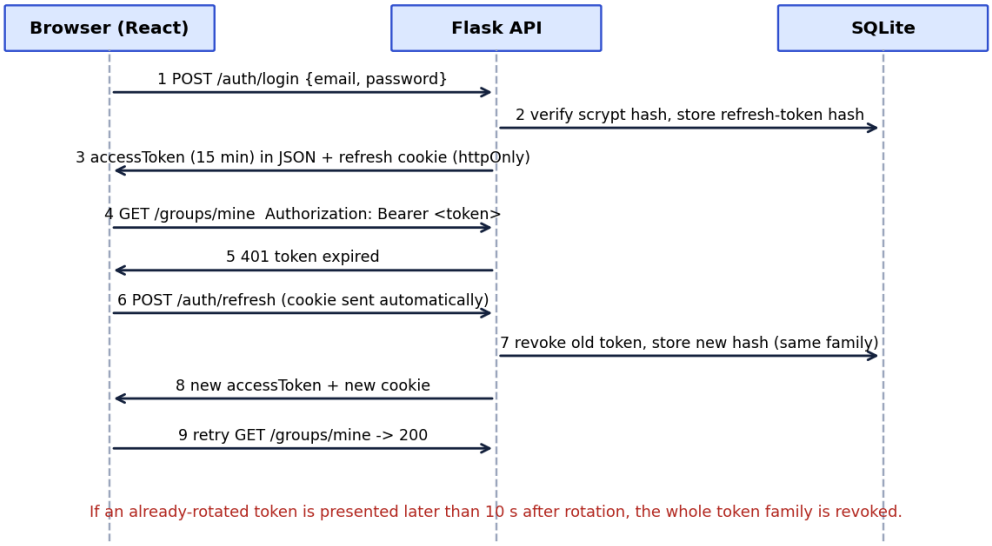



*Figure 3: Sign-in, token expiry and silent refresh.*

## 7.1 Controls implemented

| Threat | Control | Verified by |
|---|---|---|
| Stolen password database | scrypt hashes; refresh tokens stored as SHA-256 hashes | test reads the stored value and checks it is not the password |
| Guessing passwords | 10 auth requests per minute per IP; 5 failed sign-ins per 15 minutes per IP and email (one email being attacked does not lock out others); generic error for wrong email or wrong password; dummy hash check so response time does not reveal accounts | 3 rate-limit tests, 1 equal-answer test |
| Forged or tampered tokens | HS256 only (algorithm allow-list), required exp, iat and sub, token type claim, user must still exist | tests: wrong key, alg none, no expiry, wrong type, deleted user, expired |
| Stolen refresh token | httpOnly cookie scoped to /api/auth, 7-day life, rotated on every use; reuse of an old token after a 10-second grace window revokes the whole token family | tests: rotation, reuse revokes family, quick double refresh survives, expiry, logout |
| Cross-site request forgery on cookie endpoints | SameSite cookie plus an Origin allow-list check on refresh and logout; bearer token (not a cookie) authorises every data endpoint | origin test |
| Reading or changing another group's data | membership and role checked in the service for every group route; invite code only sent to members | sweep test: an outsider gets 403 on 6 read and 10 write endpoints of a private group and nothing changes |
| SQL injection | parameterised queries everywhere; LIKE wildcards escaped; column names only from fixed maps | injection strings in login, search and tags |
| Stored script content | API returns text as JSON with nosniff; the React client escapes on render; link fields accept only http and https | tests for script-tag payloads and javascript: links |
| Invite-code guessing | 6 random characters from a 32-symbol alphabet (about 10^9 codes) and 10 attempts per minute | throttle test |
| Mass assignment | only listed fields are read; registration cannot set role or id; profile cannot change email | tests |
| Abuse and leaks | 1 MB body limit, 300 requests per minute per IP, security headers (nosniff, frame deny, no-referrer, CSP default-src none, no-store, HSTS when secure), generic 500 with details only in the log | header, 413 and 500 tests |
| Weak production secret | production start-up fails unless STUDYSYNC_SECRET_KEY is set | config test |

## 7.2 Known limits

- The rate limiter and its counters live in one process; with several instances they would need Redis (risk R7).

- The access token is held in browser memory, which protects it from persistent storage theft but not from a script running inside the page. The React app never uses innerHTML, and the API sends a restrictive Content-Security-Policy for its own responses; the front-end host should add one for the app.
- No email verification or password-reset email yet (the Week 1 plan listed email as optional).

# 8. Validation and Error Handling

| Situation | Response |
|---|---|
| Field fails a rule | 422 { message: 'Please fix the highlighted fields.', errors: { field: 'Message' } } listing every bad field at once |
| Wrong type (list or object where text is expected) | 422 for that field, never a crash |
| Bad query parameter (page=abc, sort=bogus, type=video) | 422 naming the parameter; limit values above the maximum are capped |
| Body is not JSON / not an object | 415 / 400 |
| Body over 1 MB | 413 |
| Missing, invalid or expired token | 401 (expired tokens say so, which tells the client to refresh) |
| Signed in but not allowed | 403 (private group, role too low) |
| Unknown id or endpoint, wrong method | 404 / 405 as JSON, not HTML |
| Duplicate (email, membership), owner leaving | 409 |
| Too many requests | 429 with Retry-After |
| Unexpected exception | 500 with a generic message; the stack trace goes to the log only (tested) |

Validation limits match the front end: names 2 to 60 characters, group descriptions 10 to 300, titles 3 to 100, passwords 8 to 128 with a letter and a number (the upper bound stops very long passwords from being used to slow the server), tags normalised to a-z, 0-9, hyphen and underscore with at most six per resource.

# 9. Testing

Every test creates a fresh temporary database and uses separate 'actors' with their own cookie jars, so each behaves like a different browser. Password hashing is lowered to a fast setting in tests only.

| Test file | Tests | What it checks |
|---|---|---|
| test_auth.py | 33 | registration rules, duplicate emails, hashing at rest, wrong versus unknown credentials, six token-attack cases, refresh rotation, reuse detection, logout, profile and password change, login throttling |
| test_groups.py | 31 | create, private and public visibility, discovery search, subject, sort, pagination and literal % and _, SQL injection strings, join, join by code, leave, roles, removal rules, ownership transfer, one-owner constraint, cascade delete |
| test_content.py | 14 | resources (types, link validation, tag cleaning, filters, edit, delete rights, notifications), posts, comments, reply notifications, script payloads stored as text |
| test_sessions.py | 14 | staff-only scheduling, validation, offsets stored as UTC, RSVP counts and idempotence, upcoming list, notifications, dashboard counts |
| test_security.py | 20 | headers, CORS, preflight, secure cookies, JSON errors, size limit, generic 500, outsider sweep over 16 endpoints, authentication on every data route, rate limits |
| test_docs.py | 2 | every route in the code is documented and every documented route exists |
| **Total** | **114** | **all pass** (python -m unittest discover -s tests -t .) |

The front-end HTTP client was tested separately with a 12-step Node script against the running server (all steps passed). It covers login, dashboard, group creation, resource and session creation from a datetime-local value, RSVP, 403 and 422 mapping to ApiError, logout, and restoring a session after a page reload through the refresh cookie.

# 10. Performance Benchmarks

**Method.** scripts/benchmark.py seeds the demo database, starts the API on a local port (Werkzeug threaded development server, one process) and calls it over real HTTP with the requests library. Each endpoint is called 200 times in sequence. Times are wall-clock milliseconds including the HTTP round trip on loopback. A second run adds bulk data, and a third runs 20 client threads for 10 seconds. These are lab numbers from one machine with no network latency: they show the application's own cost, not what a user on a mobile connection will see. Production (gunicorn) will differ.

## 10.1 Sequential requests on the demo dataset

| Endpoint | mean | p50 | p95 | p99 |
|---|---|---|---|---|
| GET /api/health | 2.15 | 2.12 | 2.43 | 2.79 |
| GET /api/users/me | 3.96 | 3.96 | 4.46 | 4.79 |
| GET /api/users/me/dashboard | 4.34 | 4.63 | 5.13 | 5.20 |
| GET /api/groups (discover) | 3.23 | 2.80 | 5.00 | 5.18 |
| GET /api/groups?search=data | 2.97 | 2.91 | 3.55 | 3.92 |
| GET /api/groups/mine | 2.99 | 2.67 | 4.49 | 4.96 |
| GET /api/groups/{id} | 3.30 | 3.45 | 4.38 | 4.77 |
| GET /api/groups/{id}/resources | 3.05 | 2.60 | 4.39 | 5.56 |
| GET /api/groups/{id}/posts | 3.02 | 2.79 | 4.19 | 4.57 |
| GET /api/posts/{id} | 2.63 | 2.47 | 4.02 | 4.42 |
| GET /api/sessions | 3.20 | 2.94 | 4.57 | 4.89 |
| GET /api/notifications | 2.33 | 2.19 | 2.69 | 3.00 |
| PUT /api/sessions/{id}/rsvp | 3.06 | 2.77 | 4.50 | 5.69 |
| POST /api/groups/{id}/resources (writes + notifications) | 6.12 | 5.68 | 8.56 | 9.83 |

*Milliseconds, 200 requests per row. Write requests are slower because they insert the resource, its tags and one notification per other member inside a transaction.*

## 10.2 After loading a larger dataset

Bulk rows added directly in SQL: 6,006 users, 2,008 groups, 18,069 memberships, 10,210 resources and 4,004 posts (database file 6.5 MB).

| Endpoint | mean | p50 | p95 | p99 |
|---|---|---|---|---|
| GET /api/groups (discover) | 8.00 | 7.07 | 10.86 | 11.23 |
| GET /api/groups?search=graph&subject;=Databases | 4.65 | 4.34 | 6.50 | 10.30 |
| GET /api/groups?sort=newest&page;=50 | 10.39 | 9.67 | 13.00 | 13.49 |
| GET /api/groups/mine (40 groups) | 4.11 | 3.84 | 6.02 | 6.56 |
| GET /api/users/me/dashboard | 3.67 | 3.59 | 4.33 | 5.38 |
| GET /api/groups/{id}/resources | 2.74 | 2.67 | 3.37 | 4.03 |

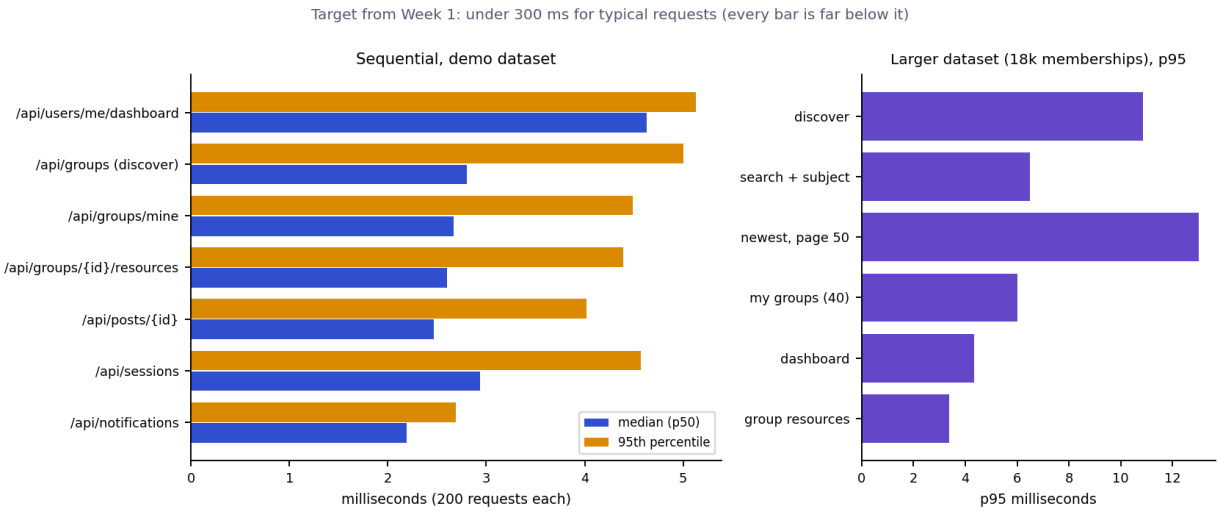



*Figure 4: Response times (left: demo data, median and 95th percentile; right: 95th percentile with the larger dataset).*

## 10.3 Sign-in cost and concurrency

| Measure | Result | Meaning |
|---|---|---|
| Sign-in (scrypt), 15 runs | mean 111.25 ms, p95 129.92 ms | Deliberately slow to make password guessing expensive; users sign in rarely |
| 20 threads, mixed reads, 10 s | 2,215 requests, 220.1 per second, 0 errors | Single process with the Python GIL; more workers under gunicorn raise this |
| Latency under that load | mean 90.31 ms, p50 86.08 ms, p95 167.33 ms, p99 206.56 ms | Waiting time with 20 clients sharing one process; still under the 300 ms target |

## 10.4 Budget check

| Metric | Target | Source | Measured | Status |
|---|---|---|---|---|
| Typical request, 95th percentile | 300 ms or less | Week 1 non-functional requirement | 5.1 ms (demo), 13.0 ms (larger data) | Pass |
| Under 20 concurrent clients, 95th percentile | 300 ms or less | Week 1 | 167.33 ms | Pass |
| List endpoints paginated | Yes | Week 1 | Discover 6 per page (max 50); in-group lists capped at 100 | Pass |
| Errors under load | 0 | Project standard | 0 | Pass |
| Full test suite time | Under 30 s | Project standard | about 5 s | Pass |

# 11. Risk Assessment

Likelihood (L) and impact (I) are scored from 1 (very low) to 5 (very high); score = L x I. High is 15 or more, Medium 8 to 14, Low 7 or less. The register covers the back end and the hand-over to deployment.

| ID | Risk | L | I | Score | Early warning | Mitigation and evidence | Status |
|---|---|---|---|---|---|---|---|
| R1 | A permission check is missed and private data is readable | 3 | 5 | 15 High | A private group's data returned to a non-member | Checks live in one place per service; sweep test over 16 endpoints; invite code hidden from non-members | Mitigated and tested |
| R2 | Refresh token or access token stolen | 3 | 5 | 15 High | Refresh used from a second place; reuse of a rotated token | 15-minute access tokens; httpOnly rotating refresh cookie; family revocation on reuse; password change ends all sessions | Mitigated; residual risk if the page itself has an XSS flaw |
| R3 | Password guessing or credential stuffing | 4 | 4 | 16 High | Spike of 401s from one IP or against one email | Per-IP and per-account limits; scrypt; generic errors; timing equalised | Mitigated; limiter is in memory (R7) |
| R4 | Invite codes guessed | 3 | 4 | 12 Medium | Many 404s from join-by-code | About 10^9 possible codes and 10 attempts per minute per IP | Mitigated |
| R5 | SQL injection | 2 | 5 | 10 Medium | Unexpected 500s or odd rows | Parameterised SQL only; escaped LIKE; fixed column maps; injection tests | Mitigated and tested |
| R6 | SQLite limits writes to one server | 3 | 3 | 9 Medium | 'database is locked' errors; need for several instances | WAL mode, short transactions, SQL kept portable; move to PostgreSQL when scaling out | Open (planned) |
| R7 | Rate-limit counters not shared across instances or restarts | 3 | 3 | 9 Medium | Limits stop working behind a load balancer | Isolated in ratelimit.py; replace with Redis counters | Open (planned) |
| R8 | Front end and API drift apart | 3 | 3 | 9 Medium | Blank sections or undefined fields after switching to the API | Responses copy the mock shapes; client smoke test; docs-versus-code test; seed data identical to the mock | Mitigated; needs the full front-end check (R12) |
| R9 | Weak or leaked secret in production | 2 | 5 | 10 Medium | Tokens forgeable if the secret leaks | Production refuses to start with the default secret; .env is git-ignored; secret read from environment | Mitigated |
| R10 | Cookie or CORS misconfiguration across sites (Vercel front end, Render API) | 3 | 4 | 12 Medium | Refresh fails with 401 in the browser only after deployment | Documented SameSite=None, Secure and origin settings; tests for secure cookie and origin allow-list | Open until deployed |
| R11 | Database file lost on a host with temporary disk | 4 | 4 | 16 High | Empty data after a redeploy | Use a persistent disk or PostgreSQL before real users; seed script restores demo data | Open (must fix before launch) |
| R12 | Front-end changes not verified in the real React build | 3 | 3 | 9 Medium | npm test or build fails, or a page breaks against the API | Changes are small and isolated to src/api; the mock still loads when VITE_API_URL is empty. Run npm test, npm run build and a manual pass | Open (do before submitting) |
| R13 | Vulnerable dependency | 2 | 4 | 8 Medium | pip audit or a security notice | Only two run-time packages (Flask, PyJWT); pin versions; run pip audit in the Week 5 pipeline (not run for this report) | Open (planned) |

Summary: 4 High (R1, R2, R3, R11), 9 Medium (R4, R5, R6, R7, R8, R9, R10, R12, R13), 0 Low. Both High risks (R1, R2) have controls with automated tests. The items still open are about deployment, not about the code: persistent storage (R11), a Redis limiter (R7) and PostgreSQL if the app outgrows one server (R6), plus the front-end verification in R12.

## 11.1 Problems found while building and how they were resolved

| Finding | Resolution |
|---|---|
| React StrictMode in development can call refresh twice at once; with rotating tokens the second call would look like token theft and sign the user out | Server grace window (10 s) for just-rotated tokens, plus single-flight refresh in the client; both tested |

| Finding | Resolution |
|---|---|
| datetime-local inputs send a time with no offset, which the server would read as UTC | The HTTP client converts to an ISO string with the browser's offset before sending; offsets are stored as UTC |
| The Week 2 session form sends the length as a string | Server accepts whole-number strings and returns numbers |

# 12. Connecting the Front End, Running and Deploying

## 12.1 Front-end integration

- The Week 2 design kept every network call in `src/api/index.js`. The Week 3 change keeps that promise: the mock moved to `mock.js`, a new `http.js` implements the same functions with fetch, and `index.js` chooses between them. Pages and components are unchanged except two small edits.
- `AuthContext.jsx`: with the real API the app starts in the 'loading' state and asks the server (through the refresh cookie) whether a session exists. `Profile.jsx`: the mock-only 'Reset demo data' section is hidden when the real API is used.
- Set `VITE_API_URL=http://localhost:5000/api` to use the API; leave it empty to use the mock (this is what the Vitest suite does, so the 23 existing tests are unaffected in principle; confirm by running them).
- Client behaviour: access token in memory only; on a 401 it refreshes once (parallel calls share one refresh) and retries; network failures become an ApiError with status 0 and a readable message; 422 field errors arrive in `err.fields` exactly as the forms expect.

## 12.2 Run locally

```
cd studysync-server
python -m venv .venv && source .venv/bin/activate     # Windows: .venv\Scripts\activate
pip install -r requirements.txt
python manage.py seed          # demo login: demo@studysync.app / Password123
python run.py                  # http://localhost:5000/api/health

python -m unittest discover -s tests -t . -v           # 114 tests
python scripts/gen_docs.py                             # rebuild docs/API.md and openapi.yaml
python scripts/benchmark.py                            # rebuild docs/benchmark.json

# front end (separate terminal, Week 2 project with the integration files copied in)
echo VITE_API_URL=http://localhost:5000/api > .env
npm install && npm run dev                             # http://localhost:5173
```

## 12.3 Deployment notes

- Run `gunicorn wsgi:app --workers 2 --threads 4`. Set STUDYSYNC_ENV=production, a long random STUDYSYNC_SECRET_KEY, STUDYSYNC_CORS_ORIGINS=your front-end URL, STUDYSYNC_COOKIE_SAMESITE=None and STUDYSYNC_COOKIE_SECURE=true (needed when the front end and API are on different sites), and STUDYSYNC_TRUST_PROXY=true behind a proxy.
- Give the SQLite file a persistent disk, or move to PostgreSQL, before real users arrive (risk R11).
- With several workers the in-memory rate limiter counts per worker; use Redis for exact limits (risk R7).

# 13. Limitations and Plan for Week 4

| Limitation | Plan |
|---|---|
| Notifications are stored but not pushed live | Add Socket.io: authenticate with the access token, one room per group and per user, emit the rows the notification service already writes; merge by id on the client to avoid duplicates |
| File resources are links | Direct upload through a storage service with size and type limits |
| No email (verification, password reset, session reminders) | Add a mail provider and a reminder job (Week 1 Could item) |
| No admin or moderation tools beyond group roles | Admin role exists in the schema; add report and remove endpoints |
| Single-process limiter and SQLite | Redis counters and PostgreSQL when moving to several instances |
| Tests run against the in-process app | Add a CI workflow (lint, tests, pip audit) as planned in Week 1 |

## 14. Conclusion

The API implements every Must and Should requirement from Week 1 that depends on a server, enforces permissions and data rules in the service layer and the database, and is covered by 114 passing tests that include attack cases for tokens, injection and cross-group access. Measured response times are far inside the 300 ms target, even with eighteen thousand memberships loaded. The honest open items are the ones that cannot be closed from a development machine: a real deployment with persistent storage, the full front-end test run, and the live Socket.io layer planned for Week 4.